Artificial Intelligence Group Assignment Report

Ticket Routing Intelligence

Explainable Multilingual Queue and Priority Prediction for Customer Support Triage

36121 Artificial Intelligence Principles and Applications

AT3 – Group 2

Group member	Student ID
Tumadhir Alsharhan	25605852
Amal Maawadh	25605864
Andres Ethorimn	26029165
Laura Pacheco	25940875
Mario Rubiano	24900627
Carl Lazarus	12043485

Repository	https://github.com/ETHLOB/trn-uts-aipa
Prototype name	Ticket Routing Intelligence
Submission date	16 May 2026

1 Executive Summary

1.1 Overview of the Project

Ticket Routing Intelligence is an AI decision-support system for customer support ticket triage. It analyses a ticket and returns predicted queue/category, priority, recommended team, escalation flag, summary, and explanation terms. The aim is not to replace agents, but to provide a structured first-pass recommendation for faster and more consistent review.

The implementation uses the Kaggle Multilingual Customer Support Tickets dataset. Five CSV files were audited; three compatible multilingual files were merged and two German-normalized files were excluded because they used a different queue taxonomy. After deduplication and filtering, the modelling dataset contains 44,275 usable tickets.

1.2 Objectives

The project objectives are:

- predict support queue/category from ticket text and safe pre-resolution metadata;
- predict priority as high, medium, or low;
- compare explainable NLP classifiers using validation and test metrics;
- benchmark multilingual transformer embeddings as an alternative representation;
- add rule-based routing and escalation logic;
- provide human-readable summaries and explanation terms;
- present the solution through a Streamlit dashboard with single-ticket and batch-ticket analysis.

1.3 Key Findings

The selected model is TF-IDF with Linear SVM for both tasks. On the held-out test set, it reached macro F1 of 0.525 for queue/category and 0.568 for priority. Billing and Payments was the strongest category, with F1 around 0.779. High priority reached F1 around 0.624. These results support first-pass triage, not full automation, so human review remains necessary.

An optional transformer-embedding benchmark used multilingual MiniLM sentence embeddings and Logistic Regression. It reached sampled macro F1 of 0.235 for category and 0.350 for priority, below the full-data TF-IDF Linear SVM pipeline. This shows that a more complex representation is not automatically better without tuning, larger samples, and careful validation.

2 Introduction

2.1 Background and Context

Customer support teams receive many unstructured tickets about billing, product questions, access, outages, returns, security, and general requests. First-pass triage is important because it decides the queue, urgency, and review path. Manual triage can be slow and inconsistent, especially for multilingual tickets or urgent cases hidden in long text.

2.2 Problem Statement

The problem is slow and inconsistent ticket triage. A ticket must be mapped to a queue, assigned priority, checked for escalation risk, and routed to the correct team. If the first decision is wrong, customers wait longer and urgent cases may be missed. This project assists the decision while keeping humans responsible for final action.

2.3 Significance and Impact

The system can reduce manual work by giving agents an initial recommendation. It also improves transparency by showing explanation terms and escalation reasons. The prototype supports single-ticket analysis and batch upload from CSV or Excel files.

2.4 Challenges and Constraints

The main constraints are dataset imbalance, multilingual text, overlapping categories, and privacy. Some languages have fewer examples, and ticket text may include personal information. The summary function masks obvious emails and phone numbers. For reproducibility and privacy, the deployed demo uses simple explainable methods instead of external LLM services.

2.5 Project Objectives and Scope

The scope is a working prototype and reproducible pipeline with notebooks, Python modules, metrics, charts, and a Streamlit app. It is not production-ready. Production use would require ticketing-system integration, access control, monitoring, retraining, stronger privacy redaction, and testing on company data.

3 Theoretical Justification

3.1 Overview of AI Methods Used

The solution combines several AI methods:

- supervised NLP classification for queue and priority prediction;
- TF-IDF vectorisation to convert text into sparse numerical features;
- Logistic Regression, Naive Bayes, and Linear SVM as comparison models;
- rule-based reasoning for routing and escalation;
- template-based summarisation with privacy masking;
- explanation terms from influential TF-IDF features;
- optional multilingual transformer embeddings for comparison evidence.

3.2 Theoretical Foundations

TF-IDF gives high weight to terms that are frequent in one document but less frequent across the corpus (Ramos, 2003). This is useful because words such as “invoice”, “refund”, “outage”, or “password” can signal a queue. Linear SVM is suitable for sparse, high-dimensional text because it learns a separating boundary between classes (Cortes and Vapnik, 1995; Joachims, 1998). Logistic Regression provides a strong linear baseline, while Naive Bayes is a simple probabilistic baseline for text (McCallum and Nigam, 1998).

The rule-based layer represents operational knowledge. For example, words such as “unauthorised”, “fraud”, or “breach” can trigger security routing or escalation even when the predicted queue is different. This is structural knowledge representation: ML predicts labels, while rules encode business risk.

3.3 Justification of Selected Methods

Linear SVM was selected because it had the best validation macro F1 for both queue and priority. It is lightweight, fast, and explainable because the app can show influential TF-IDF terms. This is more suitable for the current prototype than a heavier model that is harder to run and explain.

The transformer benchmark was kept separate from live prediction. It provides a semantic multilingual comparison using sentence embeddings (Reimers and Gurevych, 2019), but it used a sample of 2,500 tickets and performed below the current model. Therefore, it is report and future-work evidence, not the deployed prediction path.

3.4 Transformer Benchmark Rationale

The transformer benchmark used multilingual MiniLM sentence embeddings and Logistic Regression for category and priority prediction. This experiment addresses SLO4 by testing a modern deep-learning representation related to transformer, Generative AI, and LLM methods. MLP, reinforcement learning, and LLM assistant ideas were discussed by the group, but only the transformer benchmark is present in the current main repository. The live demo keeps TF-IDF because it is faster and stronger in the measured results.

3.5 Related Work / Literature Support

The project follows standard NLP classification practice: text normalisation, TF-IDF feature extraction, train/validation/test splitting, model comparison, and held-out evaluation. The responsible AI design follows human oversight, leakage control, transparency, and privacy minimisation (Ribeiro, Singh and Guestrin, 2016; Jobin, Ienca and Vayena, 2019).

4 Workflow and Methodology

4.1 System Architecture Overview

The workflow has seven stages: audit data, prepare text, build features, train models, evaluate results, route tickets, and explain outputs. Streamlit presents this workflow in the Overview tab and provides prediction tools in the Try Solution tab.

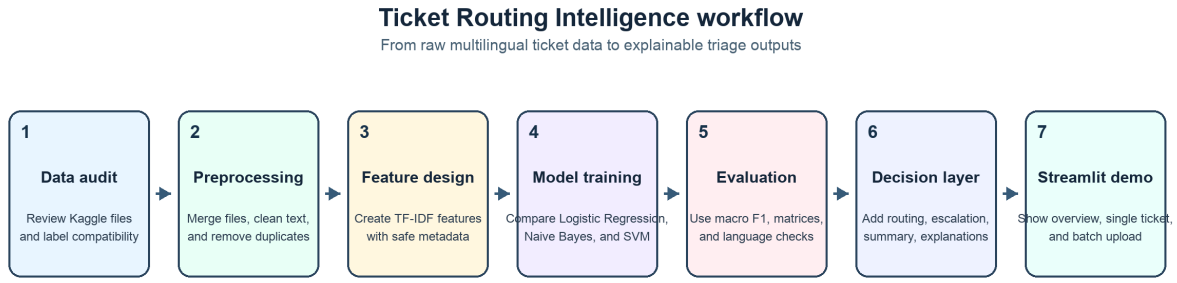


Figure 1: Project workflow used in the final prototype.

4.2 Data Collection and Preprocessing

The dataset was downloaded from Kaggle. The compatible files were:

- `aa_dataset-tickets-multi-lang-5-2-50-version.csv`
- `dataset-tickets-multi-lang-4-20k.csv`

- `dataset-tickets-multi-lang3-4k.csv`

The two German-normalized files were audited but excluded because their queue taxonomy was different.

The pipeline normalises column names, combines subject and body into `ticket_text`, removes duplicate subject-body pairs, and removes rows without usable text. It excludes `answer` because that field is written after the agent responds, so using it would create target leakage.

4.3 Feature Engineering

The main feature is `model_text`, which combines cleaned ticket text with safe metadata tokens such as type, language, and tags. Cleaning lowercases text, masks emails and phone numbers, replaces long numbers, removes URLs, and keeps simple alphanumeric tokens. TF-IDF uses unigrams and bigrams with up to 30,000 features.

4.4 Model Design and Algorithmic Approach

The project trains two separate supervised tasks:

- queue/category prediction using the dataset queue label;
- priority prediction using the dataset priority label.

For each task, Logistic Regression, Multinomial Naive Bayes, and Linear SVM were compared. The data was split into training, validation, and test sets. Models were selected using validation macro F1, retrained on training plus validation data, and evaluated on the held-out test set.

4.5 Integration of AI Paradigms

The final prototype integrates supervised learning, a probabilistic baseline, symbolic rules, explainability, template summarisation, and transformer comparison. This shows multiple AI paradigms. Only the stable TF-IDF Linear SVM pipeline is used for live predictions.

4.6 Design Decisions vs Implementation Details

A key design decision was to prioritise reproducibility and explainability. The app does not call an external LLM; summaries are template-based and explanations use TF-IDF terms. This reduces cost, privacy risk, and dependency risk. Transformer results are shown in the Overview tab but are not used for routing.

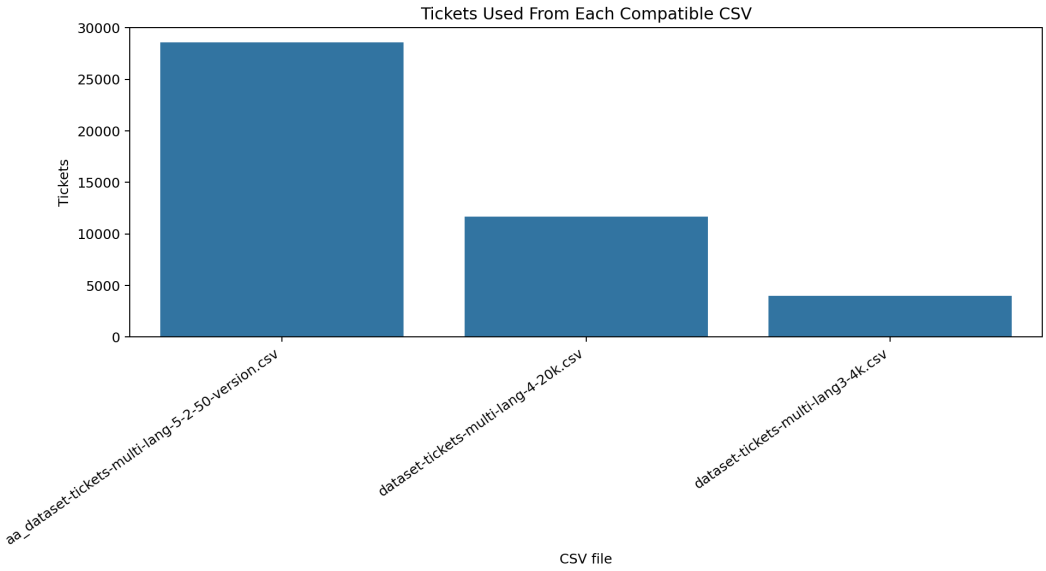
5 Empirical Analysis and Results

5.1 Experimental Setup

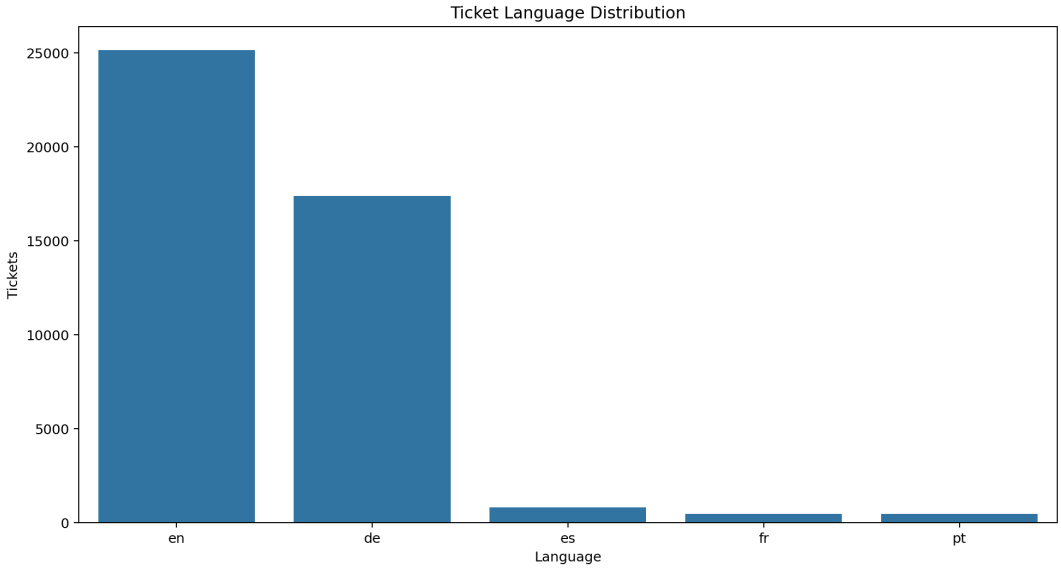
The final modelling frame contains 44,275 rows. The pipeline uses train/validation/test splitting with stratification when possible and a fixed random seed for reproducibility. Metrics are saved in `results/`; charts are saved in `report_assets/`.

5.2 Dataset Description

The data includes English, German, Spanish, French, and Portuguese tickets. English and German dominate the dataset. Labels include Billing and Payments, Product Support, Technical Support, Customer Service, Service Outages and Maintenance, Returns and Exchanges, Sales and Pre-Sales, Human Resources, and General Inquiry.

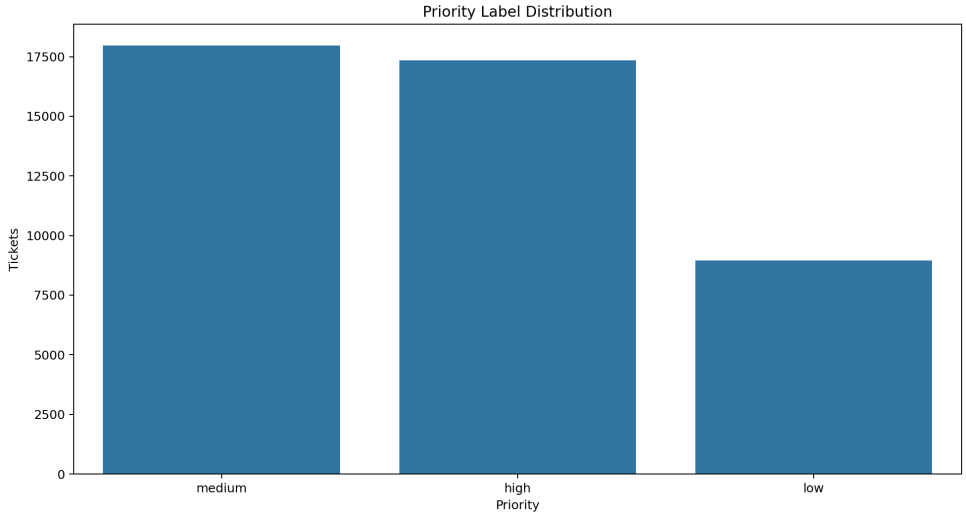


(a) Rows contributed by each compatible Kaggle CSV file.

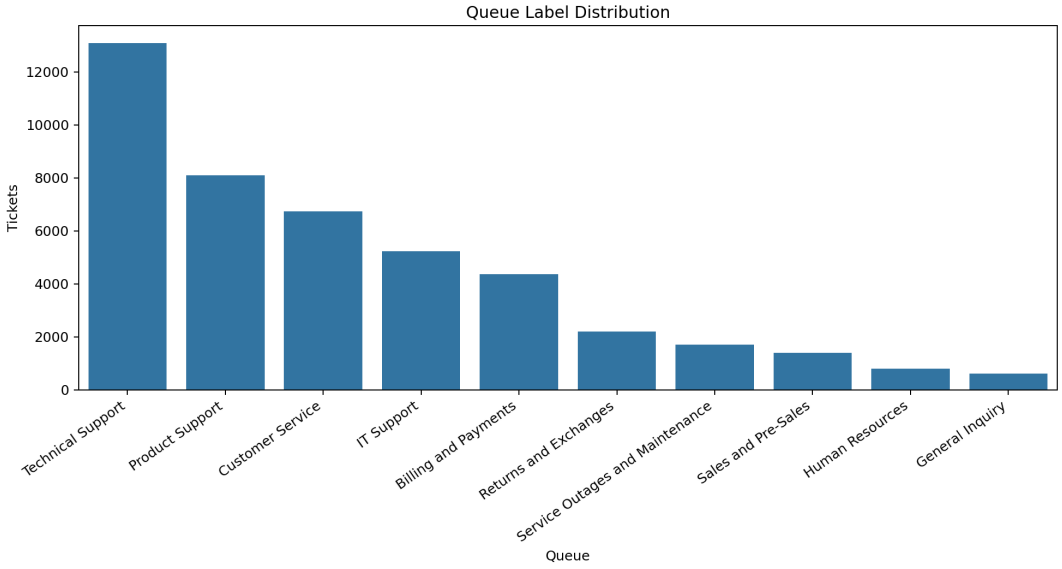


(b) Ticket language distribution after preprocessing.

Figure 2: Dataset composition used by the final modelling pipeline.



(a) Priority distribution.



(b) Queue/category distribution.

Figure 3: Target-label distributions for priority and queue prediction.

5.3 Evaluation Metrics

The project reports accuracy, macro precision, macro recall, macro F1, and weighted F1. Macro F1 is the main selection metric because it gives equal importance to each class, which is important for imbalanced data.

5.4 Results

Task	Model	Split	Accuracy	Macro P	Macro R	Macro F1
Category	Linear SVM	Test	0.531	0.507	0.547	0.525
Priority	Linear SVM	Test	0.581	0.567	0.569	0.568
Category	Transformer + LR	Sampled	0.272	0.249	0.293	0.235
Priority	Transformer + LR	Sampled	0.357	0.359	0.359	0.350

Table 1: Main test results and optional transformer benchmark results.

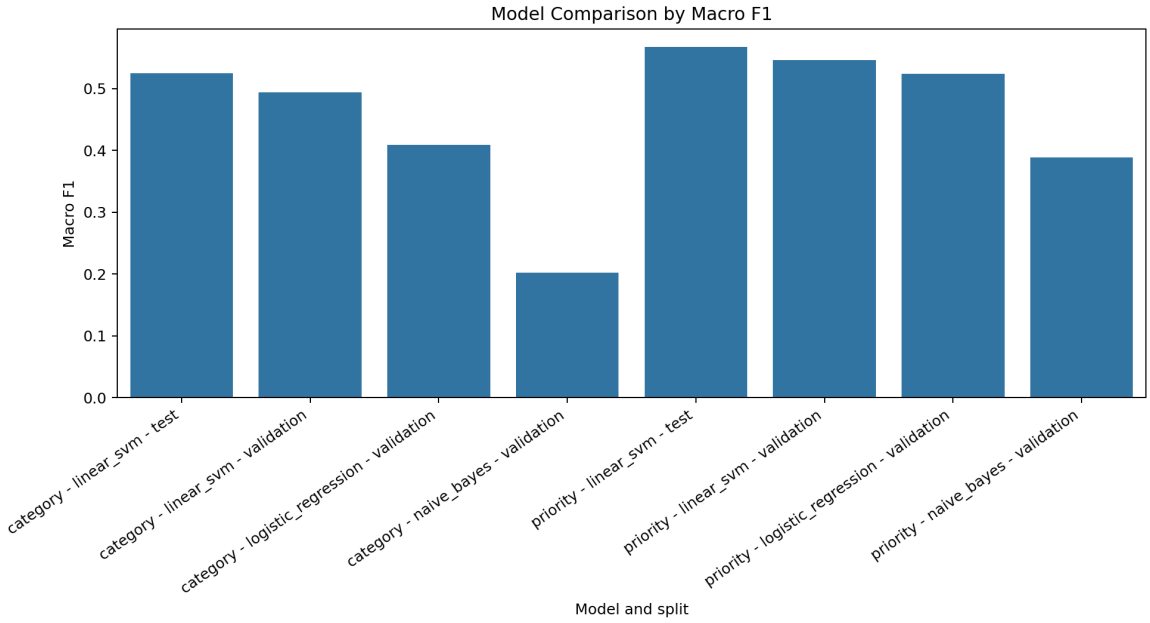


Figure 4: Validation and test macro F1 comparison for the main models.

5.5 Transformer Benchmark Results

The optional transformer benchmark used a reproducible sample of 2,500 tickets. Multilingual MiniLM embeddings were classified with Logistic Regression. Category reached 0.235 sampled macro F1 and priority reached 0.350, below the full-data TF-IDF Linear SVM results of 0.525 and 0.568.

Task	Representation and classifier	Macro F1	Accuracy
Category	TF-IDF + Linear SVM, full test set	0.525	0.531
Category	Multilingual MiniLM embeddings + Logistic Regression, sampled test	0.235	0.272
Priority	TF-IDF + Linear SVM, full test set	0.568	0.581
Priority	Multilingual MiniLM embeddings + Logistic Regression, sampled test	0.350	0.357

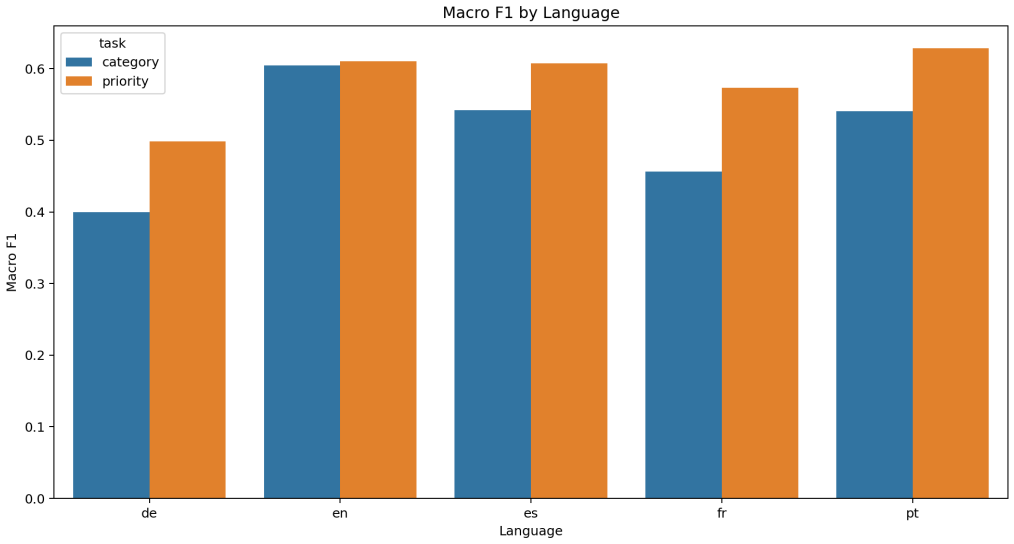
Table 2: Direct report comparison between the deployed TF-IDF model and the optional transformer benchmark.

This does not mean transformers are unsuitable. It means this sampled and lightly tuned experiment did not outperform the simpler pipeline. A stronger setup would need more data, class balancing, hyperparameter tuning, and possibly a better classifier. For this assignment, the result supports keeping Linear SVM for live prediction and discussing transformers as validated future work.

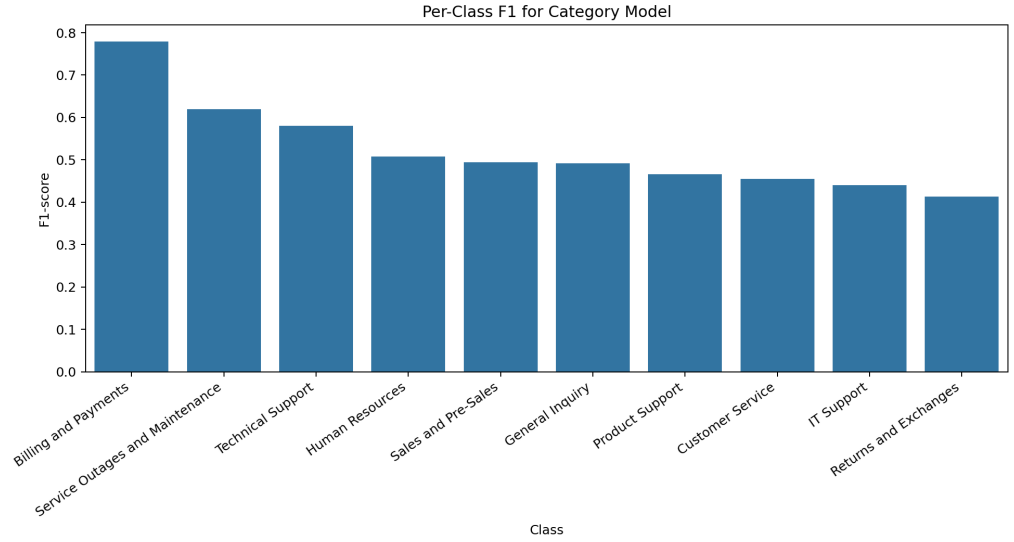
5.6 Result Interpretation and Discussion

The results show useful predictive signal but also limitations. Billing and Payments is strongest because billing tickets have distinctive vocabulary. Service Outages and Maintenance also performs well. Product Support, IT Support, and Customer Service are harder because wording overlaps. Priority prediction is slightly stronger than category prediction, but low priority remains weaker than high and medium.

Language analysis shows a performance gap: English category macro F1 is 0.604, while German category macro F1 is 0.400. Future work should improve multilingual preprocessing, use more balanced data, or tune multilingual embeddings more carefully.



(a) Macro F1 by language and task.



(b) Per-class category F1.

Figure 5: Detailed performance views used for critical reflection.

6 Critical Reflection

6.1 Limitations of the Approach

The main limitations are moderate macro F1, language imbalance, training/inference mismatch, and limited privacy masking. Training uses text plus safe metadata, while manually pasted tickets mainly provide text. Batch files can include richer columns. The app masks obvious emails and phone numbers, but it does not detect all personal information.

6.2 Failure Cases

Some tickets are ambiguous. A security issue may mention billing details and be predicted as Billing and Payments. The recommended team can still be Security Team because routing also uses security terms. This is useful, but it must be explained to users.

6.3 Trade-offs

The main trade-off is explainability versus complexity. TF-IDF Linear SVM is simple, fast, and explainable. Transformer embeddings can capture deeper semantic meaning, but they performed worse in the sampled benchmark and require heavier dependencies. LLM summarisation could produce richer text, but it adds privacy, cost, and reproducibility risks.

6.4 Scalability and Computational Constraints

The prototype runs locally with saved model files. Production use would need an API, authentication, logging, monitoring, drift detection, retraining, and ticketing-platform integration. Larger transformer experiments may need better hardware or cloud resources.

6.5 Alternative Methods and Improvements

Future work should include confidence calibration, active learning from agent corrections, Unicode-aware preprocessing, stronger PII redaction, SLA-aware escalation, and better transformer experiments. A text-only model could reduce the single-ticket training/inference mismatch.

7 GitHub Repository

7.1 Repository Link

The GitHub repository is:

https://github.com/ETHLOB/trn-uts-aipa

7.2 Instructions to Run the Project

Create and install the environment:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install -e .
```

Download the Kaggle files:

```
.\.venv\Scripts\python.exe -m customer_support_ai.download_data
```

Train and evaluate the models:

```
.\.venv\Scripts\python.exe -m customer_support_ai.train
.\.venv\Scripts\python.exe -m customer_support_ai.report_assets
```

Run the Streamlit app:

```
.\.venv\Scripts\streamlit.exe run app/streamlit_app.py
```

Optional transformer benchmark:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-transformer.txt
.\.venv\Scripts\python.exe -m customer_support_ai.transformer_benchmark --sample-size
↪  2500
```

8 Conclusion

8.1 Summary of Findings

This project built an AI decision-support prototype for customer support triage. It uses a real multilingual dataset, reproducible preprocessing, supervised model comparison, held-out evaluation, routing rules, escalation logic, explanation terms, and a Streamlit dashboard.

8.2 Key Insights

The strongest current approach is TF-IDF with Linear SVM. It gives moderate but useful performance and supports explainability. Results are strongest where categories have clear vocabulary and weaker where categories overlap or language representation is limited. The transformer benchmark did not improve performance, so the final design keeps the simpler model for live predictions.

8.3 Future Work

Future work should improve multilingual handling, add confidence scores, learn from human corrections, strengthen privacy redaction, and test transformer, multi-layer network, reinforcement learning, or LLM-based methods under fairer conditions. Production deployment would also need monitoring, data governance, and support-system integration.

9 Individual Contributions

The table below summarises individual contributions using repository history, presentation artefacts, and WhatsApp evidence. Discussed work not present in the current main repository is marked as pending evidence.

Member	**Confirmed contribution in repository or local artefacts**	**Pending contribution evidence**
Tumadhir Alsharhan ID: 25605852	Led the main AI pipeline: notebooks 01–05, reusable modules, Streamlit app, README, checklist, result files, charts, and demo examples. Amal supported planning and report/presentation framing. Tumadhir also prepared presentation and Q&A material.	Confirmed in repository history, WhatsApp evidence, and presentation artefacts.
Mario Rubiano ID: 24900627	Reviewed code, notebooks, app, outputs, and requirements. Added executed notebook evidence and a status/gap report. Implemented the transformer benchmark, notebook 06, result files, dependency file, documentation updates, and Streamlit benchmark display. Prepared responsible AI/future work content.	Confirmed in repository history and presentation artefacts.
Andres Ethorimn ID: 26029165	Created the repository/template, shared GitHub access, maintained main, merged pull requests, cleaned large files, restored result artefacts, reviewed Mario's PR, and presented empirical results in slide notes.	Confirmed in repository history, WhatsApp evidence, and presentation artefacts.
Amal Maawadh ID: 25605864	Supported Tumadhir in project planning and implementation direction. Shared the implementation plan covering dataset, workflow, models, metrics, demo features, and HD requirements. Drafted presentation content and notes, and presented the introduction/problem framing.	Confirmed by WhatsApp evidence and presentation artefacts.

Member	Confirmed contribution in repository or local artefacts	Pending contribution evidence
Laura Pacheco ID: 25940875	Presented the solution overview, dataset explanation, and leakage-control explanation. WhatsApp evidence indicates EDA involvement and experimentation with an MLP priority model and reinforcement learning routing agent.	**PENDING:** MLP Neural Network and reinforcement learning routing agent are mentioned in chat, but code/results are not present in current main repository content. Add branch output or keep as future work.
Carl Lazarus ID: 12043485	Presented the AI methods section. WhatsApp evidence indicates that Carl added a demo slide/video and refined the demo wording.	**PENDING:** The LLM AI assistant for ticket submission is mentioned as branch work, but code is not present in current main repository content. Add branch output or keep as future work.

10 References

1. Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
2. Joachims, T. (1998). Text categorization with support vector machines: learning with many relevant features. *ECML*.
3. McCallum, A. and Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. *AAAI Workshop on Learning for Text Categorization*.
4. Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. *Proceedings of the First Instructional Conference on Machine Learning*.
5. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*.
6. Ribeiro, M. T., Singh, S. and Guestrin, C. (2016). Why should I trust you? Explaining the predictions of any classifier. *KDD*.
7. Jobin, A., Ienca, M. and Vayena, E. (2019). The global landscape of AI ethics guidelines. *Nature Machine Intelligence*, 1, 389–399.
8. scikit-learn developers. (2026). scikit-learn: Machine Learning in Python. https://scikit-learn.org/
9. Streamlit developers. (2026). Streamlit documentation. https://docs.streamlit.io/
10. Kaggle. (2026). Multilingual Customer Support Tickets dataset. https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets

A Application Screenshots

This appendix provides visual evidence of the Streamlit prototype and its main user-facing functions.

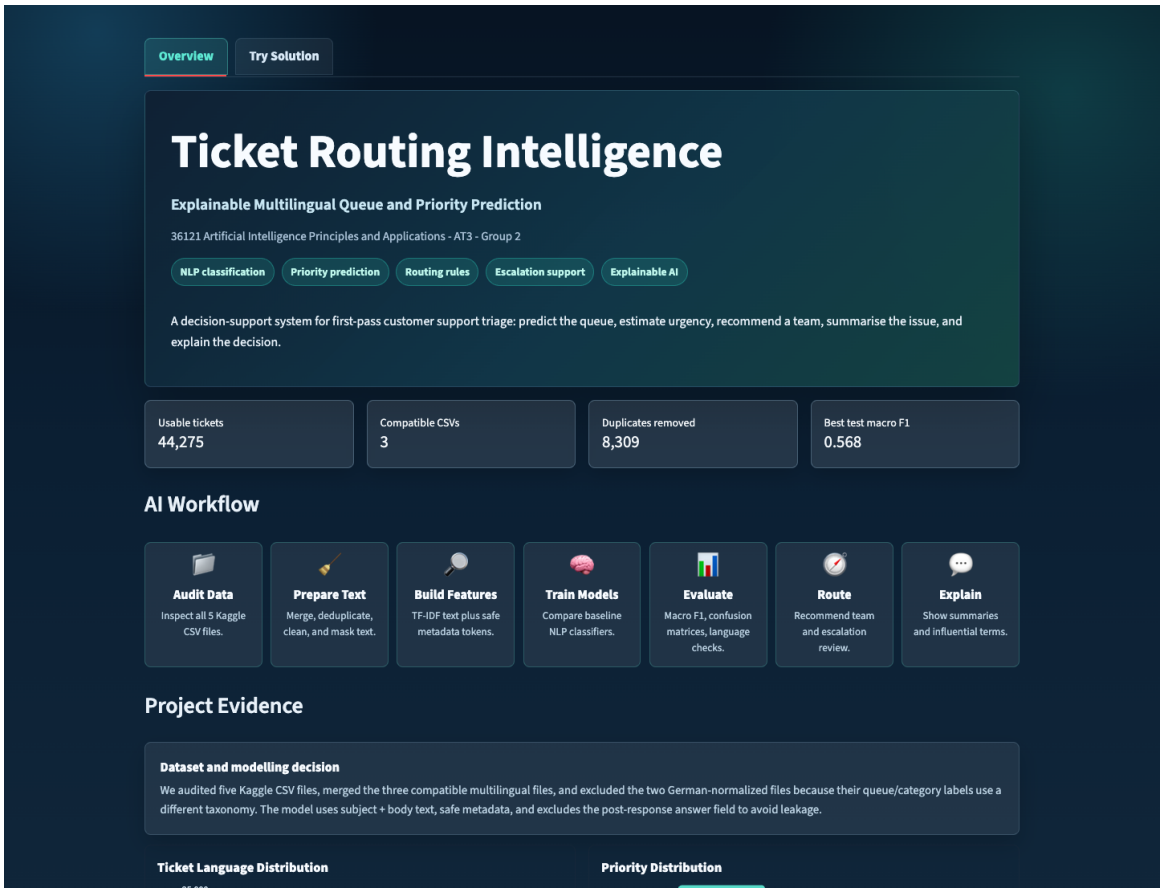


Figure 6: Overview dashboard with project summary, metrics, workflow, and evidence panels.

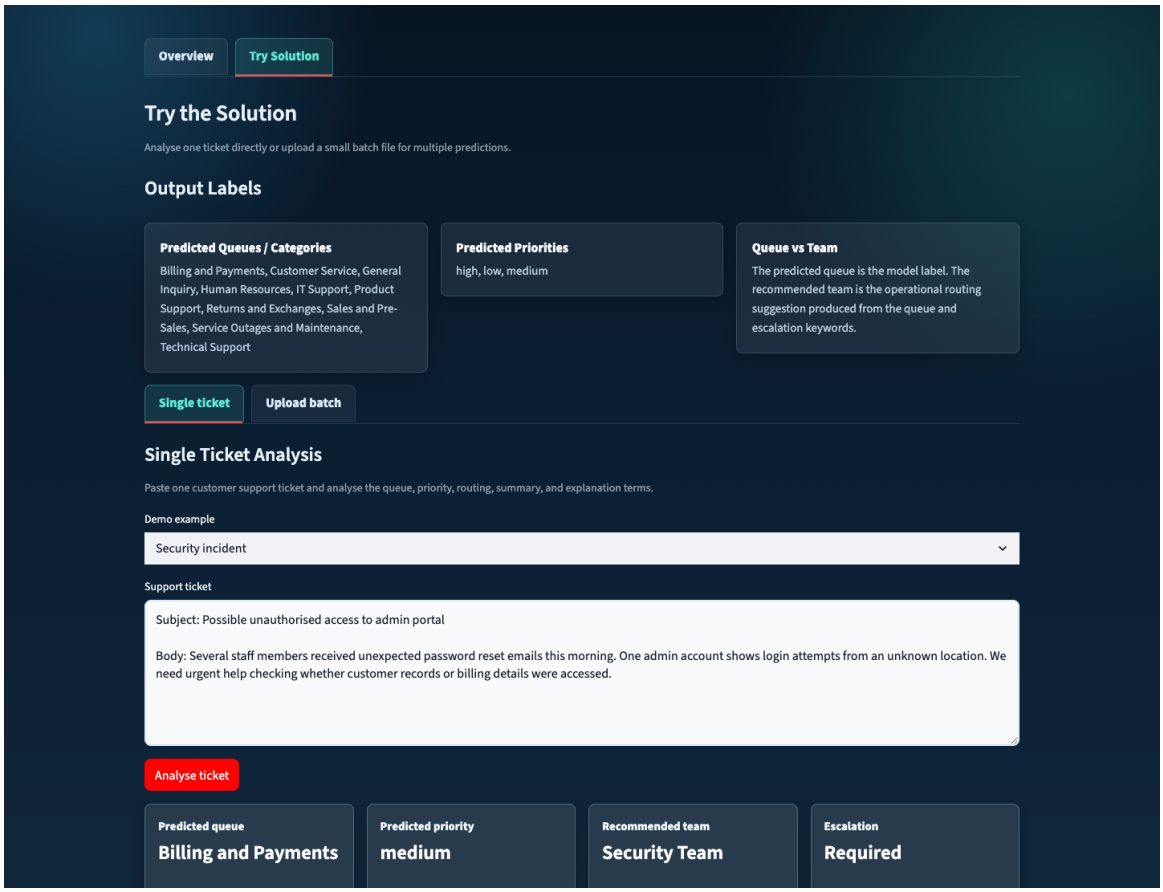


Figure 7: Single-ticket analysis showing queue, priority, recommended team, and escalation output.

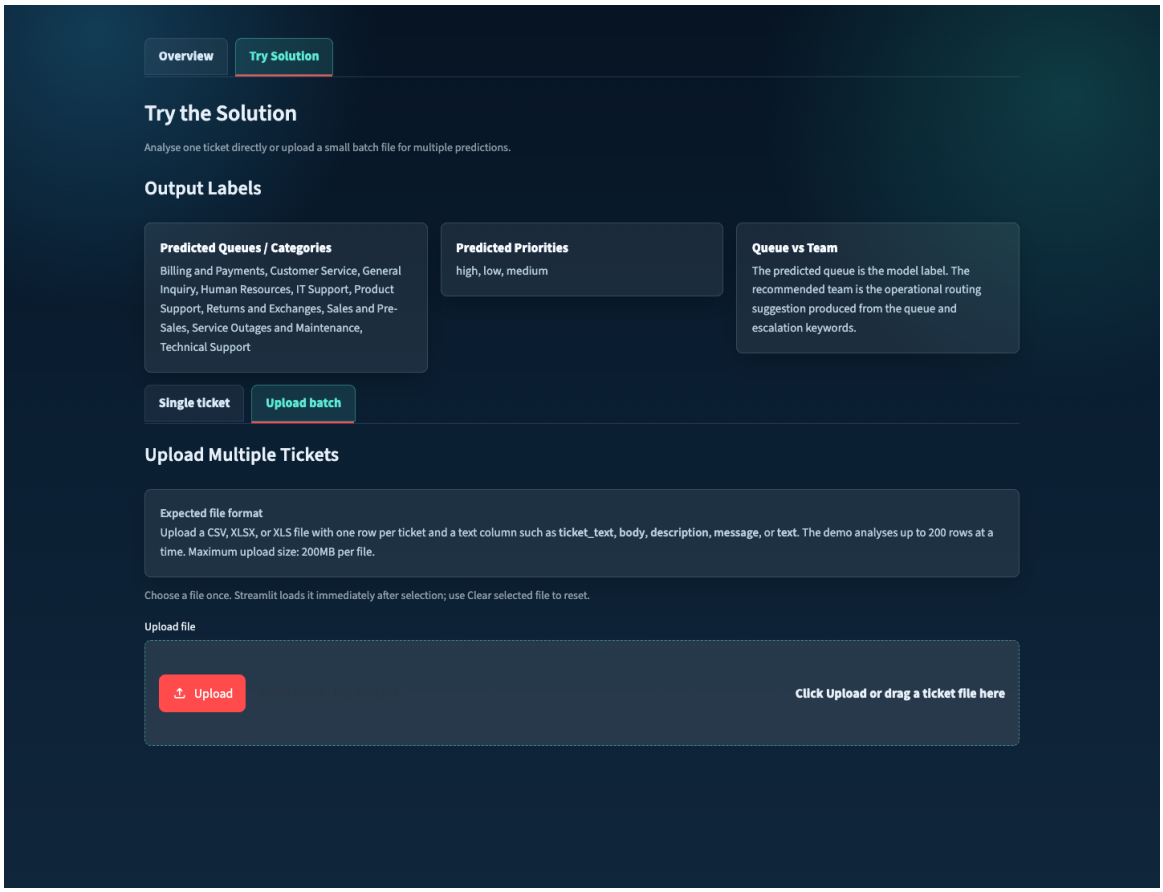


Figure 8: Batch upload interface for CSV or Excel ticket files.